

What Software Engineering Can Learn from Soccer

Shari Lawrence Pfleeger

We've talked about software quality for a long time, developing numerous software quality assurance approaches in the hope of making our software increasingly better. Charles Mann, contributing editor of MIT's *Technology Review*, points out that other technologies—televisions, cars, airplanes, bridges—have improved over time as their engineering matured; he asks why software has not. In the February 2001 issue of *Communications of the ACM*, Edsger Dijkstra said that software's biggest challenge is “how not to make a mess of it.” So, where have we gone wrong?



To answer this question, we can look at how other disciplines learn and grow. Software development is as much an art as a science, and we learn lessons from both perspectives. Many of us think of ourselves as engineers: we train in engineering departments and rely on engineering tools, techniques, and quantitative methods to guide us. But our work's artistic side—which those who promote agile methods often emphasize—plays an important role, too. As good software developers, we are grounded in artistic engineering activities such as modeling and design. Our good people skills enable us to work with customers and on teams. And we need good instincts to select the best approaches and products to use.

Appreciating instinct

Instinctual expertise is hard to develop and

difficult to trust, because it sometimes seems far afield from the engineering approaches we are taught. But the decision-making literature suggests that we should appreciate the essential role of instinct in the three kinds of expertise Jens Rasmussen describes (*Information Processing and Human-Machine Interaction: An Approach to Cognitive Engineering*, North-Holland, 1986): skill-based, rule-based, and knowledge-based.

Skill-based

This expertise is heavy on instinct, using highly integrated, automatic sensory and motor responses; it takes little conscious effort to apply these skills. We develop similar expertise in sports or music. We learn particular movements—hitting a tennis ball a certain way or playing scales repeatedly—until they become imprinted and we can do them instinctively. Effectiveness at this level depends on our innate skill, our experience, and how much we practice. It also depends on a predictable environment; if we raise the net or change the configuration of the strings or keys, our skills are less certain and possibly less useful.

In software development, we might become skill-based testers by looking for the same kinds of faults in the same kinds of code until spotting those faults becomes second nature. Similarly, working many times on similar projects in similar circumstances could help us generate excellent cost estimates.

Rule-based

By contrast, when we use rule-based expertise, we are consciously aware of the

steps we take to attain a goal. Not always explicitly formulated, the steps are somehow codified into loose or strict rules that we follow in a given situation. For example, good software designers have rules about checking interfaces, including exception handling, and addressing performance issues. We might follow these rules without even realizing it before we turn a design over to inspectors or programmers. As with skill-based expertise, these rules rely on familiarity with the situation and its context. In a novel situation or an unfamiliar environment, a rule-based expert might need to perform additional analysis.

Knowledge-based

A knowledge-based expert recognizes goals, makes them explicit, and then develops procedures to reach them. The recognizing and procedure-building could be automatic, in that the expert seems to know instinctively what to do and when to do it. Once he or she decides on a grand plan, the approach becomes rule-based, where the procedures include steps to tackle each aspect of the overall problem. We see knowledge-based expertise when a good designer proposes a software architecture. The designer instinctively knows that one framework, say pipe-and-filter, will be more effective than another and why. Once the overall architecture is laid out, the designer follows ingrained rules about the best ways to document and improve it.

Software and soccer

Instinct is vital in developing and using our expertise. To learn to understand and develop those instincts, let us look at soccer, a game that blends art and science. Clearly some soccer players are more talented than others and some are better in certain roles than others. A coach aims to improve players' talents and blend them to play a good game. The players are much like software developers, and the coach resembles a team leader or project manager. But looking deeper, we learn more.

Professional soccer players do not

just show up for games, play them, and leave the stadium. Instead, they participate in three important ways: they perfect basic skills, analyze and abstract from past performances, and build a set of patterns that eventually becomes instinctual. These three aspects of performance analysis can help software engineers improve software's quality. To see why, first consider our basic skills. Good software development often depends on good modeling and design. But if we train software engineers at all, we usually train them to write programs first and to think about modeling and design later—after bad habits are already embedded in how we do business. Instead of focusing first on programming, we can learn how to model and design. We can also learn how to use different modeling and design approaches together and in context to determine what a system will do and how it will do it. That is, we learn which techniques are appropriate in which contexts so that doing good modeling and design become second nature.

Next, consider what happens at the end of the game. At the final whistle or bell, the players do not grab their belongings and head for home. Instead, they join the coach in the locker room to learn from the day's play. They might watch videos of the game or simply recount to each other what they did and why. As they

discuss their moves, the reasons for them, and the success or failure of the plays, the coach builds abstractions—diagrams with Xs and Os to show what transpired. The players learn to see the game not only from their own perspectives but also from the perspective of others' roles and responsibilities. The abstractions capture the essence of each play and become a set of patterns players can choose from in future games. Eventually, the players act as a team—invoking appropriate patterns based on past experience—rather than remaining a collection of individuals who happen to meet on the field.

In software, we rarely perform such post-project analysis. We pack our bags and leave for the next project without stopping to analyze what we did well and what we did poorly. We build no abstractions of the requirements, designs, code, or tests to help us learn what was most effective. We create no templates or checklists to capture the best of our analysis and review. And we do not use abstractions or patterns to drill and make instinctual the best actions and activities.

Modeling and abstraction are key elements in quality improvement, and we want those skills to become instinctual. We cannot improve our products and processes if we do not stop to absorb lessons from the past. Gary Klein's research on decision-making under pressure (*Sources of Power: How People Make Decisions*, MIT Press, 1998) tells us that the best decision-makers are those with rich and diverse "mental databases" of metaphors about which actions are most appropriate in which circumstances. Unless we take the time to analyze and abstract and to build our lessons learned into skill-, rule-, and knowledge-based expertise, we will never have the mental and technical tools to keep from "making a mess of it." ☛

Professional soccer players do not just show up for games, play them, and leave the stadium.

Shari Lawrence Pfleeger is a senior researcher at RAND. Contact her at shari_pfleeger@rand.org.